

Модульное программирование

в языке программирования C++

Проблемы, возникающие при работе над крупными проектами:

- Время компиляции.
- Организация коллективной работы над проектом.
- Грамотная организация исходного кода.
- Повторное использование собственного и стороннего кода.

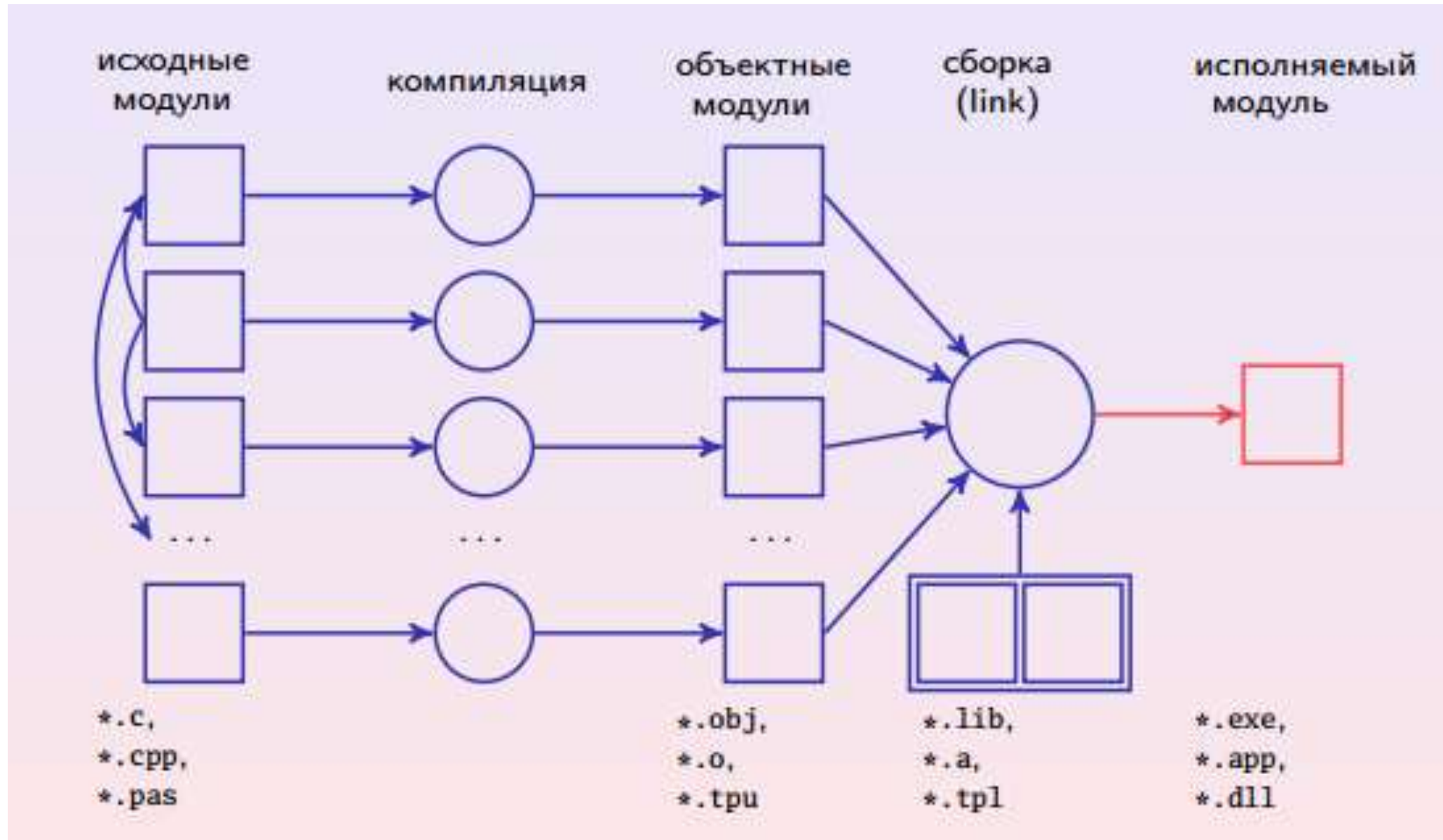
Технология модульного программирования включает

- поддержку со стороны языка программирования;
- набор правил для разработчиков.

Модулем обычно называют совокупность связанных функций и тех данных, которыми они управляют.

- Разделение программы на функции позволяет избежать избыточности кода, т.к. функцию записывают один раз, а вызывать ее на выполнение можно многократно из разных точек программы.
- Часто используемые функции можно помещать в **библиотеки**.
- Следующим шагом в повышении уровня абстракции программы является группировка функций и связанных с ними данных в **отдельные файлы (модули)**, компилируемые отдельно.
- Получившиеся в результате компиляции объектные модули объединяются в исполняемую программу с помощью **компоновщика**.
- Разбиение на модули уменьшает время перекомпиляции и облегчает процесс отладки, скрывая несущественные детали за интерфейсом модуля и позволяя отлаживать программу по частям (или разными программистами).

Схема модульного подхода



Правила создания модулей

- Для ускорения компиляции и сборки проекта, а также для упрощения повторной используемости модулей необходимо уменьшать количество зависимостей между модулями (в т. ч., от стандартных библиотек).
- Необходимо, чтобы каждый модуль был **как можно более универсален**.
- В каждый модуль необходимо помещать **логически взаимосвязанные описания**.

- **Объявление (declaration)** — вводит новые имена в транслируемый модуль или повторно вводит имена, введённые в предыдущих объявлениях.
- Может быть одновременно **определением (definition)** этих имён.
- **Область действия (scope)** — для имени — часть программы, в которой оно может использоваться и означает одно и то же.
- **Связь (linkage)** — свойство имени, позволяющее означать то же самое в другой области, если в ней дать его (повторное) объявление.

Модульность в языке C++

Модуль в C++ логически состоит из двух файлов:

- **заголовочного файла (header file)** – с расширением **.h**

Он включает прототипы функций, объявление переменных и констант, о которых должны знать остальные модули, подключение необходимых для реализации заголовочных файлов.

- **файла с исходным кодом (source file)** – с расширением **.cpp**, который содержит определение функций, а также определения глобальных переменных и констант (если они есть).

Применение модульного программирования в C++

f.cpp

```
// ...  
  
void f()  
{  
    // ...  
}
```

f.h

```
void f();
```

g.cpp

```
#include "f.h"  
  
// ...  
  
void g()  
{  
    // ...  
    f();  
    // ...  
}
```

Правила создания заголовочных файлов

- Помещаются объявления и определения, необходимые в нескольких транслируемых модулях.
- Стандартное расширение (h, hpp).
- В начале и конце вставляются директивы препроцессора, предотвращающие включение содержимого файла при его повторном подключении:

Пример

```
/*  
    mydefs.h: My definitions  
*/  
  
#ifndef MYDEFS_H__  
#define MYDEFS_H__  
  
/* Здесь идут некоторые объявления и определения */  
  
#endif    /* MYDEFS_H__ */
```

Объявления и определения должны быть **идентичными** для всех транслируемых модулей

Пример (b.h)

```
struct Data
{
    int m_n1;
#ifdef INCLUDED_FROM_B_CPP
    int m_n2, m_n3, m_n4;
#endif
};
void f(Data d);
```

b.cpp

```
#define INCLUDED_FROM_B_CPP
#include "b.h"

void f(Data d)
{
    // ...
    d.m_n4 = 4;
}
```

a.cpp

```
#include "b.h"

int main()
{
    Data d;
    f(d);
    // ...
}
```

Правила создания заголовочных файлов

- В одном файле должны группироваться **взаимозависимые** (возможно, логически) **объявления и определения**, которые, скорее всего, будут все одновременно нужны (или почти все) в некотором транслируемом модуле.
- Иначе **разные объявления и определения** разумнее поместить в **разных файлах**.

Объекты, помещаемые, в заголовочные файлы

- Определения констант.
- Определения встроенных функций (**inline**).
- Объявления шаблонов, для которых не нужно полное определение.

```
const double cdPi = 3.1415926;
```

```
inline void Inc(int &rn)
{
    ++ rn;
}
```

```
template <class T>
    struct Hidden;

template <class T>
    struct Data
    {
        // ...
        Hidden <T> *m_pHidden;
    };
};
```

Объекты, помещаемые, в заголовочные файлы

- Определения всех остальных шаблонов
- Пространства имён (**namespace**, могут быть вложены) кроме макроопределений.

```
template <class T>
    struct Hidden;

template <class T>
    struct Data
    {
        // ...
        Hidden <T> *m_pHidden;
    };

```

```
namespace mylib
{
    const double cdPi = 3.1415926;
    //
    double my_fast_sin(double d);
    //
    // ...
}

```

Правила для пространств имён:

- Пространство имён **std** зарезервировано для стандартной библиотеки, в нём **нельзя ничего объявлять**.
- Не принято помещать в заголовочные файлы директивы **using (using namespace x;)**, также в ограниченных случаях следует использовать объявления **using (using x::f;)**.